

스프링 처음 접할 때 테스트해볼 쉬운 예제 모음

목적

스프링 게시판을 만들기 전에, 학생들이 먼저 아래 흐름을 가볍게 경험하게 한다.

브라우저 주소 입력
↓
Spring Controller 실행
↓
문자열 / JSON 반환
↓
HTML에서 fetch로 API 호출
↓
화면에 출력

build.gradle

```
plugins {  
    id 'java'  
    id 'io.spring.dependency-management' version '1.1.7'  
}  
  
group = 'com.example'  
version = '0.0.1-SNAPSHOT'  
  
java {  
    toolchain {  
    }  
}  
  
repositories {  
}
```

게시판에 바로 들어가면 Controller, JSON, fetch, @RequestBody, List<Board>가 한 번에 나와서 어렵다.

그래서 먼저 작은 예제들로 감각을 만든 뒤 게시판으로 연결한다.

dependencies {
 implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
 implementation 'org.springframework.boot:spring-boot-starter-web' // starter-webmvc 대신 표준 웹 라이브러리 사용

compileOnly 'org.projectlombok:lombok'
 annotationProcessor 'org.projectlombok:lombok'

testImplementation 'org.springframework.boot:spring-boot-starter-test' // 통합 테스트 라이브러리
 testCompileOnly 'org.projectlombok:lombok'
 testRuntimeOnly 'org.junit.platform:junit-platform-launcher'

}

```
tasks.named('test') {  
    useJUnitPlatform()  
}
```

1. 서버 켜지고 주소 접속해보기

목표

스프링 프로젝트가 실행되고 브라우저에서 응답이 오는지 확인한다.

예제 코드

```
package com.example.hello;
```

```
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
```

```
public class HelloController {
```

```
    @GetMapping("/")
```

```
    public String home() {
```

```
        return "스프링부트 서버 실행 성공!";
```

```
    }
```

```
}
```

```
application.properties
```

```
# 1. 서버 포트 지정 (기본값인 8080을 명시적으로 세팅)  
server.port=8080
```

```
# 2. 하이버네이트 스키마 자동 마이그레이션 기능 완전히 끄기 (에러 방지)  
spring.jpa.hibernate.ddl-auto=none
```

```
# 3. 콘솔창에 SQL 쿼리 로그가 예쁘게 찍히도록 설정  
spring.jpa.show-sql=true  
spring.jpa.properties.hibernate.format_sql=true
```

```
# 4. (중요) 임시로 메모리 데이터베이스(H2) 연결 설정하기  
# 실제 MySQL이나 Oracle 주소를 적기 전까지는 이 가상 DB 설정이 있어야 에러가 안 납니다.
```

```
spring.datasource.url=jdbc:h2:mem:testdb  
spring.datasource.driver-class-name=org.h2.Driver  
spring.datasource.username=sa  
spring.datasource.password=
```

확인 주소

```
http://localhost:8080
```

설명

```
@RestController
```

브라우저나 JavaScript에게 데이터를 응답하는 컨트롤러다.

```
@GetMapping("/")
```

/ 주소로 GET 요청이 들어오면 아래 메서드를 실행한다.

해볼 문제

1. 출력 문구를 자기 이름으로 바꿔보기
2. /hello 주소를 추가해서 "안녕하세요" 출력하기
3. /school 주소를 추가해서 학교 이름 출력하기

예시 답

```
@GetMapping("/hello")
public String hello() {
    return "안녕하세요!";
}
```

2. 주소마다 다른 글자 출력하기

목표

@GetMapping 주소 개념을 익힌다.

예제 코드

```
@GetMapping("/about")
public String about() {
    return "여기는 소개 페이지입니다.";
}
```

```
@GetMapping("/contact")
public String contact() {
    return "연락처 페이지입니다.";
}
```

확인 주소

http://localhost:8080/about 
http://localhost:8080/contact 

해볼 문제

1. /food 주소를 만들고 좋아하는 음식 출력하기
2. /game 주소를 만들고 좋아하는 게임 출력하기
3. /today 주소를 만들고 오늘 기분 출력하기

예시 답

```
@GetMapping("/food")
public String food() {
    return "제가 좋아하는 음식은 돈까스입니다.";
}
```

3. 숫자 계산 결과 출력하기

목표

자바 메서드 결과가 브라우저에 출력되는 것을 확인한다.

예제 코드

```
@GetMapping("/add")
public String add() {
    int a = 10;
    int b = 20;
    int result = a + b;

    return "결과: " + result;
}
```

확인 주소

http://localhost:8080/add

해볼 문제

1. /minus 주소에서 30 - 10 결과 출력하기
2. /multi 주소에서 7 * 8 결과 출력하기
3. /score 주소에서 국어, 영어, 수학 평균 출력하기

예시 답

```
@GetMapping("/score")
public String score() {
    int kor = 80;
    int eng = 90;
    int math = 100;

    int avg = (kor + eng + math) / 3;

    return "평균 점수: " + avg;
}
```

4. 주소에 값 넣어보기: @PathVariable

목표

주소에 들어온 값을 자바 변수로 받는다.

예제 코드

```
@GetMapping("/hello/{name}")
public String helloName(@PathVariable String name) {
    return name + "님 안녕하세요!";
}
```

확인 주소

```
http://localhost:8080/hello/민수
http://localhost:8080/hello/지우
```

설명

```
/hello/민수
  ↑
  이 부분이 name 변수에 들어감
```

해볼 문제

1. /food/치킨 → "내가 좋아하는 음식은 치킨입니다."
2. /age/18 → "나이는 18살입니다."
3. /square/5 → "5의 제곱은 25입니다."

예시 답

```
@GetMapping("/square/{num}")
public String square(@PathVariable int num) {
    return num + "의 제곱은 " + (num * num) + "입니다.";
}
```

게시판 연결

나중에 게시판 상세 조회에서 사용한다.

```
@GetMapping("/api/boards/{id}")
public Board getBoard(@PathVariable Long id) {
    ...
}
```

5. 쿼리스트링 사용해보기: @RequestParam

목표

주소 뒤에 `?name=홍길동` 같은 값을 받는다.

예제 코드

```
@GetMapping("/welcome")
public String welcome(@RequestParam String name) {
    return name + "님 환영합니다!";
}
```

확인 주소

http://localhost:8080/welcome?name=홍길동

설명

?name=홍길동
↑
name 변수에 홍길동이 들어감

해볼 문제

1. /introduce?name=민수&age=18
→ "민수님의 나이는 18살입니다."
2. /menu?food=돈까스
→ "오늘의 추천 메뉴는 돈까스입니다."
3. /calc?a=10&b=20
→ "합계는 30입니다."

예시 답

```
@GetMapping("/calc")
public String calc(@RequestParam int a, @RequestParam int b) {
    return "합계는 " + (a + b) + "입니다.";
}
```

6. 객체를 JSON으로 반환해보기

목표

`@RestController` 가 객체를 JSON으로 바꿔주는 것을 확인한다.

Student 클래스

```
package com.example.demo;

public class Student {
    private String name;
    private int score;

    public Student(String name, int score) {
        this.name = name;
        this.score = score;
    }

    public String getName() {
        return name;
    }

    public int getScore() {
        return score;
    }
}
```

컨트롤러 코드

```
@GetMapping("/student")
public Student student() {
    return new Student("김자바", 90);
}
```

브라우저 결과

```
{
  "name": "김자바",
  "score": 90
}
```

설명

자바에서는 `Student` 객체를 반환했지만, 브라우저에서는 JSON 형태로 보인다.

Java 객체
↓
Spring이 자동 변환
↓
JSON

해볼 문제

1. Movie 클래스를 만들고 title, rating 반환하기
2. Food 클래스를 만들고 name, price 반환하기
3. Game 클래스를 만들고 title, genre 반환하기

7. 객체 여러 개를 JSON 배열로 반환해보기

목표

`List<T>` 가 JSON 배열이 되는 것을 확인한다.

예제 코드

```
import java.util.ArrayList;
import java.util.List;

@GetMapping("/students")
public List<Student> students() {
    List<Student> list = new ArrayList<>();

    list.add(new Student("김자바", 90));
    list.add(new Student("이자바", 80));
    list.add(new Student("박자바", 70));

    return list;
}
```


브라우저 결과

```
[
  {
    "name": "김자바",
    "score": 90
  },
  {
    "name": "이자바",
    "score": 80
  },
  {
    "name": "박자바",
    "score": 70
  }
]
```

설명

Student → 학생 1명
List<Student> → 학생 여러 명

게시판에서는 이렇게 연결된다.

Board → 게시글 1개
List<Board> → 게시글 여러 개

해볼 문제

1. 영화 3개를 List<Movie>로 반환하기
2. 음식 3개를 List<Food>로 반환하기
3. 게시글 3개를 List<Board>로 반환하기

8. 간단한 HTML 화면 띄우기

목표

`static/index.html` 이 자동으로 열리는 것을 확인한다.

파일 위치

```
src/main/resources/static/index.html
```

예제 코드

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>스프링 첫 화면</title>
</head>
<body>
  <h1>스프링부트 첫 화면</h1>
  <p>static 폴더의 index.html입니다.</p>
</body>
</html>
```

확인 주소

```
http://localhost:8080
```

설명

Spring Boot는 `src/main/resources/static/index.html` 파일을 자동으로 첫 화면으로 보여준다.

해볼 문제

1. 자기소개 화면 만들기
2. 좋아하는 음식 3개 목록 만들기
3. 버튼 하나 추가하기
4. CSS로 배경색 바꾸기

PPT 20

9. HTML에서 Spring API 호출해보기

목표

`fetch()` 로 스프링 API를 호출하는 첫 경험을 한다.

컨트롤러 코드

```
@GetMapping("/api/message")
public String message() {
    return "스프링에서 온 메시지입니다!";
}
```

index.html 코드

```
<button onclick="loadMessage()">메시지 가져오기</button>
<p id="result"></p>

<script>
function loadMessage() {
    fetch("/api/message")
        .then(response => response.text())
        .then(data => {
            document.getElementById("result").innerText = data;
        });
}
</script>
```

설명

```
버튼 클릭
↓
fetch("/api/message")
↓
Spring Controller 실행
↓
문자열 응답
↓
화면에 출력
```

해볼 문제

1. /api/today API 만들기
2. 버튼을 누르면 오늘 기분 가져오기
3. /api/menu API 만들기
4. 버튼을 누르면 추천 메뉴 가져오기

10. JSON API를 HTML에 출력해보기

목표

JSON 객체를 받아서 화면에 출력한다.

컨트롤러 코드

```
@GetMapping("/api/student")
public Student apiStudent() {
    return new Student("김자바", 90);
}
```

index.html 코드

```
<button onclick="loadStudent()">학생 정보 가져오기</button>
```

```
<div id="student-box"></div>
```

```
<script>
```

```
function loadStudent() {
```

```
    fetch("/api/student")
```

```
    .then(response => response.json())
```

```
    .then(data => {
```

```
        document.getElementById("student-box").innerHTML = `
```

```
            <h3>${data.name}</h3>
```

```
            <p>점수: ${data.score}</p>
```

```
        `;
```

```
    });
```

```
}
```

```
</script>
```

각 조에서 가장 웃긴 응답 문구 만들기

콘솔 먼저 찍어본 뒤 innerHTML
console.log(data);
console.log("이름"+data.name);
console.log("성적"+data.score);

설명

```
response.json()
```

서버에서 받은 JSON 응답을 JavaScript 객체처럼 사용할 수 있게 꺼낸다.

해볼 문제

1. Movie JSON을 화면에 출력하기
2. Food JSON을 화면에 출력하기
3. Game JSON을 화면에 출력하기

11. JSON 배열을 HTML 목록으로 출력해보기

목표

게시판 목록 출력 직전 연습을 한다.

컨트롤러 코드

```
@GetMapping("/api/students")
public List<Student> apiStudents() {
    List<Student> list = new ArrayList<>();

    list.add(new Student("김자바", 90));
    list.add(new Student("이자바", 80));
    list.add(new Student("박자바", 70));

    return list;
}
```

index.html 코드

```
<button id="loadStudents" onclick="loadStudents()">학생 목록 가져오기</button>
<div id="student-list"></div>

<script>
function loadStudents() {
    fetch("/api/students")
        .then(response => response.json())
        .then(data => {
            const listBox = document.getElementById("student-list");
            listBox.innerHTML = "";

            data.forEach(student => {
                const item = document.createElement("div");
            });
        });
    console.log(data);
}
```

id=loadStudents
onclick="loadStudents()"
출력할 곳
const loadBtn = document.getElementById("loadStudents");
loadBtn.addEventListener("click", function(){
loadStudents();
});
console.log(data);

+ = 으로

```
item.innerHTML = `
  <h3>${student.name}</h3>
  <p>점수: ${student.score}</p>
`;

listBox.appendChild(item);
});
});
}
</script>
```

설명

data.forEach(...)

JSON 배열 안에 있는 데이터를 하나씩 꺼내서 화면에 붙인다.

해볼 문제

0. 게시판 목록 3개 출력하기 : 깃허브에 소스

1. 영화 목록 3개 출력하기
2. 음식 목록 3개 출력하기
3. 게임 목록 3개 출력하기
4. 점수가 80점 이상이면 "우수" 표시하기

게시판 연결

나중에 게시글 목록은 이렇게 출력한다.

```
data.forEach(board => {
  const boardItem = document.createElement("div");

  boardItem.innerHTML = `
    <div class="title">[${board.id}] ${board.title}</div>
    <div class="content">${board.content}</div>
  `;

  listContainer.appendChild(boardItem);
});
```

12. POST 요청 맛보기

목표

브라우저에서 JSON을 서버로 보내본다.

컨트롤러 코드

DTO 없이 간단히 `Map`으로 받는다.

```
import java.util.Map;

@PostMapping("/api/echo")
public Map<String, String> echo(@RequestBody Map<String, String> data) {
    return data;
}
```

index.html 코드

```
<input type="text" id="message" placeholder="메시지 입력">
<button onclick="sendMessage()">보내기</button>

<div id="result"></div>

<script>
function sendMessage() {
    const message = document.getElementById("message").value;

    fetch("/api/echo", {
        method: "POST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify({
            message: message
        })
    })
    .then(response => response.json())
    .then(data => {
        document.getElementById("result").innerText =
            "서버가 받은 메시지: " + data.message;
    });
}
</script>
```

설명

```
JS 객체
↓
JSON.stringify()
↓
JSON 문자열
↓
@RequestBody
↓
Java Map으로 받음
```

해볼 문제

1. name을 보내고 서버가 다시 name을 반환하게 하기
2. title, content를 보내고 다시 출력하기
3. price를 보내고 "가격: 1000원" 출력하기

게시판 연결

나중에는 이렇게 바뀐다.

```
@PostMapping
public Board createBoard(@RequestBody Board board) {
    return boardRepository.save(board);
}
```

처음 접하는 학생용 추천 순서

스프링 첫날에 전부 다 하지 말고 아래 순서로 끊어서 진행한다.

1. "/" 주소에서 문자열 반환
2. "/hello/{name}"으로 PathVariable 연습
3. "/calc?a=10&b=20"으로 RequestParam 연습
4. Student 객체 JSON 반환
5. List<Student> JSON 배열 반환
6. static/index.html 띄우기
7. HTML에서 fetch로 API 호출
8. JSON 배열을 화면에 출력

- 9. POST 요청 맛보기
- 10. 게시판 등록/조회로 연결

게시판 들어가기 전 꼭 해보면 좋은 미니 실습 3개

미니 실습 1. 학생 목록 API

목표:

Spring API가 JSON을 반환하는 것 이해

미니 실습 2. HTML에서 학생 목록 출력

목표:

fetch + response.json() + forEach 이해

미니 실습 3. 메시지 POST 보내기

목표:

JSON.stringify + @RequestBody 이해

이 3개만 제대로 해도 게시판 등록/조회 이해도가 확 올라간다.